

מילון מונחים:

- "עץ" - עץ AVL
- עדר מלא/עדר לא ריק - עדר שיש בו לפחות סוס אחד
- *master* - הסוס שעוקבים אחריו
- *slave* - הסוס שעוקב אחרי סוס אחר

תיאור של מבני הנתונים

התכנית מורכבת מ-4 מחלקות עיקריות: *Plains, Herd, Horse, AVLtree*.

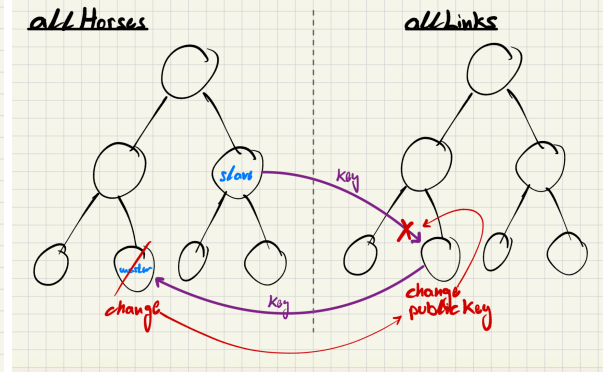
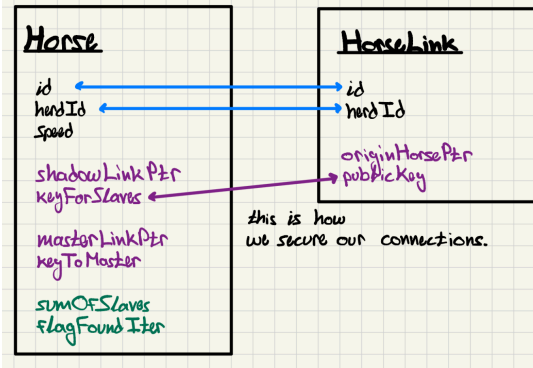
מחלקת *AVLtree* מממשת עץ AVL כפי שנלמד בהרצאה.

מחלקת *Horse* מכילה:

- *id*
- *speed*
- *id - herdId* של העדר אליו הסוס משתייך עם ערך דפולטיבי 0
- מחלקת עזר *HorseLink* שמהווה שכבת אימות עבור *Horse* בעזרתה אנו מונעים גישה לא חוקית לזיכרון. הסבר נוסף ודוגמאות בפעולות *can_run_together, leads, join_herd*
- *shadow* - מצביע ל-*HorseLink* המתאים לסוס
- *masterLink* - מצביע ל-*HorseLink* המתאים ל-*master* של הסוס
- *keyForSlaves* - הערך שנשמר אצל ה-*slaves* של הסוס עם ערך דפולטיבי 0 (כאשר לסוס אין *slaves*)
- *keyToMaster* - הערך של *keyForSlaves* של ה-*master* של הסוס עם ערך דפולטיבי 0 (כאשר לסוס אין *master*)
- *sumOfSlaves* ו-*flagFound* - נדרשים למימוש של *can_run_together*

מחלקת *HorseLink* מכילה:

- *originHorse* - מצביע לסוס
- *id* של הסוס
- *herdId* של הסוס
- *publicKey* - *keyForSlaves* של הסוס

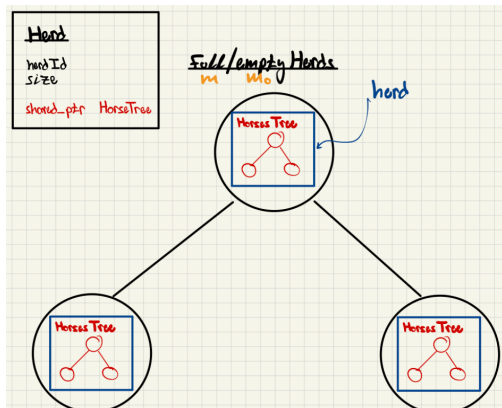
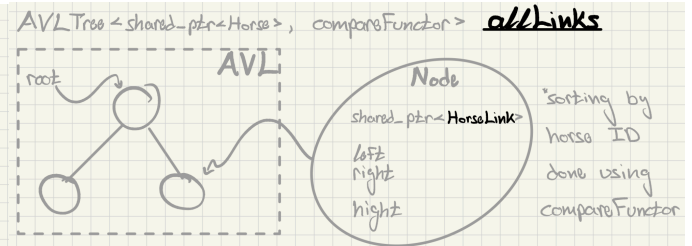
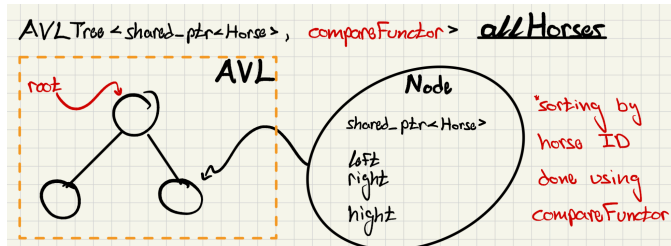


מחלקת *Herd* מכילה:

- *herdId*
- *size* - מספר הסוסים בעדר
- *horseTree* - עץ של סוסים

מחלקת *Plains* מכילה:

- *allHorses* - עץ מסוג *Horse* של כל הסוסים בתוכנית ממיון לפי *id*
- *allLinks* - עץ מסוג *HorseLink* של כל ה-*HorseLinks* בתוכנית ממיון לפי *id*
- *fullHerds* - עץ מסוג *Herd* של כל העדרים הלא ריקים בתוכנית ממיון לפי *herdId*
- *emptyHerds* - עץ מסוג *Herd* של כל העדרים הריקים בתוכנית ממיון לפי *herdId*



סיבוכיות מקום

סימונים לצורכי סיבוכיות:

נסמן ב- n את מספר הסוסים במערכת.

ב- m את מספר העדרים שאינם ריקים במערכת.

ב- $m_\emptyset$ את מספר העדרים הריקים במערכת.

בתוכנית ישנם מספר משתנים מקומיים עם סיבוכיות מקום $O(1)$, לכן הם לא משפיעים על הסיבוכיות הכוללת. ישנם 4 עצים - עץ של כל הסוסים ועץ של כל ה-*HorseLinks*, כלומר לכל היותר $2n$ אובייקטים, ועץ של כל העדרים הלא ריקים עם לכל היותר m אובייקטים ועץ של כל העדרים הריקים עם לכל היותר $m_\emptyset$ אובייקטים. בנוסף כל אחד מעצי העדרים מכיל עצים של מצביעים לסוסים השייכים לעדר. כל תתי העצים האלה מכילים את כל n הסוסים פעם אחת, כי לא ייתכן מצב שבו סוס שייך למספר עדרים בו זמנית. בפעולה *can_run_together* אנו מקצים 2 מחסניות זמניות של סוסים בגודל n לכל היותר, ככמות הסוסים. בסה"כ יש לנו לכל היותר $O(1) + m_\emptyset + m + 4n$ אובייקטים, לכן סיבוכיות המקום הכוללת של התוכנית היא $O(n + m + m_\emptyset)$ כנדרש.

מימוש התוכנית

• *plains_t()*

מאתחל את 4 העצים שהמחלקה *plains* מכילה (סוסים, *HorseLinks*, עדרים ריקים, עדרים לא ריקים) כאשר הם ריקים.

סיבוכיות זמן: $O(1)$ במקרה הגרוע.

• $\sim plains_t()$

מוחק את העצים על ידי קריאה לפונקציה *delete* של כל עץ.

סיבוכיות זמן: כפי שנלמד בהרצאה סיבוכיות פעולת המחיקה חסומה על ידי כמות האיברים בעץ. הראינו קודם כי סיבוכיות המקום של כל העצים ביחד הינה $O(n + m + m_\emptyset)$, לכן גם סיבוכיות הזמן של ההורס היא $O(n + m + m_\emptyset)$ במקרה הגרוע.

• *add_herd(int herdId)*

נבדוק שמזהה העדר תקין ושלא קיים עדר נוסף, ריק או לא ריק, עם המזהה הזה על ידי קריאה לפונקציה *find* של העץ. אם הבדיקה צלחה נוסיף את העדר לעץ העדרים הריקים על ידי קריאה לפונקציה *insert* של העץ.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים חיפוש בשני עצי העדרים, פעולה זו היא $O(\log m) + O(\log m_\emptyset)$ במקרה הגרוע. לאחר מכן אנו מוסיפים איבר לעץ העדרים הריקים, פעולה זו היא $O(\log m_\emptyset)$ במקרה הגרוע. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log m + \log m_\emptyset)$ במקרה הגרוע.

• `remove_herd(int herdId)`

נבדוק שממזה העדר תקין ושהעדר נמצא בעץ העדרים הריקים על ידי קריאה לפונקציה `find`. לאחר מכן נוציא את העדר על ידי קריאה לפונקציה `remove` של העץ.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים חיפוש בעץ העדרים הריקים, פעולה זו היא $O(\log m_\emptyset)$ במקרה הגרוע. לאחר מכן אנו מוציאים איבר מעץ העדרים הריקים, פעולה זו היא $O(\log m_\emptyset)$ במקרה הגרוע. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log m_\emptyset)$.

• `add_horse(int horseId, int speed)`

נבדוק שהערכים תקינים ונוודא שלא קיים סוס עם אותו המזהה על ידי חיפוש בעץ כל הסוסים בעזרת `find`. אם הבדיקה צלחה נוסיף את הסוס לעץ כל הסוסים בעזרת `insert`. כעת ניצור `HorseLink` של הסוס החדש ונכניס אותו לעץ כל ה-`HorseLinks` בעזרת `insert`. כמו כן נעדכן את השדה המתאים אצל הסוס.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים חיפוש בעץ כל הסוסים, פעולה זו היא $O(\log n)$ במקרה הגרוע. לאחר מכן אנו מוסיפים איברים לעצי כל הסוסים וכל ה-`HorseLinks`, פעולות אלה הן $O(\log n)$ במקרה הגרוע. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log n)$.

• `join_herd(int horseId, int herdId)`

נבדוק שהערכים תקינים ונוודא שהסוס והעדר קיימים על ידי חיפוש בעצי כל הסוסים, כל העדרים הריקים והמלאים. אם הבדיקה צלחה נבדוק שהערך של `herdId` של הסוס עדיין דפולטיבי, מה שמסמל שהסוס עדיין לא שייך לעדר כלשהו. לאחר מכן נעדכן את `herdId` של הסוס ונכניס אותו לעדר המתאים. כעת נרצה לוודא שהסוס מתנתק מה-`master` שלו ומנתק את כל ה-`slaves` שלו. לשם כך נאפס את `keyToMaster` ונוסיף 1 ל-`keyForSlaves` כדי שה-`slaves` לא יוכלו לגשת אליו יותר. כמו כן נעדכן את `publicKey` ואת `herdId` ב-`HorseLink`. לבסוף אם העדר היה ריק לפני כן נעביר אותו לעץ העדרים הלא ריקים.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים חיפוש בעצי כל הסוסים, כל העדרים הריקים והמלאים בסיבוכיות $O(\log n + \log m + \log m_\emptyset)$ במקרה הגרוע ומכניסים סוס לעדר בסיבוכיות $O(\log n)$ במקרה הגרוע, כי יש בו לכל היותר n סוסים. עדכון שדות הוא $O(1)$. העברת

העדר לעץ העדרים הלא ריקים כוללת הוצאה מעץ העדרים הריקים והכנסה לעץ העדרים הלא ריקים, כלומר היא $O(\log m + \log m_\emptyset)$ במקרה הגרוע. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log n + \log m + \log m_\emptyset)$ במקרה הגרוע.

• *follow(int horseId, int horseToFollowId)*

נבדוק שהערכים תקינים, שהסוסים קיימים וששניהם באותו עדר. אם ה-*master* לא היה *master* לפני כן נעדכן את *keyForSlaves* שלו כדי לסמן שהוא *master* ואז נעדכן את *keyToMaster* ואת *masterLink* אצל ה-*slave*.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים חיפוש בעץ כל הסוסים בסיבוכיות $O(\log n)$ במקרה הגרוע. עדכון שדות הוא $O(1)$. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log n)$ במקרה הגרוע.

• *leave_herd(int horseId)*

נבדוק שה-*id* תקין, נוודא שהסוס קיים, שהוא שייך לעדר כלשהו ושהעדר הזה אכן קיים. נאפס את *herdId* אצל הסוס ונוציא אותו מהעדר. כעת נרצה להתנתק מה-*master* ולנתק את כל ה-*slaves* של הסוס. לשם כך נאפס את *keyToMaster* ונוסיף 1 ל-*keyForSlaves* כדי שה-*slaves* לא יוכלו לגשת אליו יותר. כמו כן נעדכן את *publicKey* ואת *herdId* ב-*HorseLink*. לבסוף אם העדר התרוקן נעביר אותו לעץ העדרים הריקים.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים חיפוש בעצי כל הסוסים, כל העדרים הריקים והמלאים בסיבוכיות $O(\log n + \log m + \log m_\emptyset)$ במקרה הגרוע. מוציאים את הסוס מהעדר ב- $O(\log n)$ במקרה הגרוע (אם הוא מכיל את כל הסוסים). עדכון שדות הוא $O(1)$. העברת העדר לעץ העדרים הריקים כוללת הוצאה מעץ העדרים הלא ריקים והכנסה לעץ העדרים הריקים, כלומר היא $O(\log m + \log m_\emptyset)$ במקרה הגרוע. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log n + \log m + \log m_\emptyset)$ במקרה הגרוע.

• *get_speed(int horseId)*

נבדוק שה-*id* תקין, נמצא את הסוס בעץ כל הסוסים ונחזיר את מהירותו.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים חיפוש בעץ כל הסוסים ב- $O(\log n)$ במקרה הגרוע ומחזירים ערך ב- $O(1)$. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log n)$ במקרה הגרוע.

• *leads(int horseId, int otherHorseId)*

נבדוק שהערכים תקינים, נמצא את שני הסוסים ונוודא שהם קיימים ונמצאים באותו עדר. נגדיר את הסוס הראשון להיות "הסוס הנוכחי", נבדוק אם ה- id שלו שווה ל- $otherHorseId$ ונסמן אותו בדגל כדי לדעת שעברנו עליו. אם כן - הצלחנו. אם לא - נבדוק אם ל"סוס הנוכחי" יש $master$ באופן הבא:

- נוודא שיש לסוס $masterLink$ (ה- $HorseLink$ של ה- $master$)
- נוודא שה- $publicKey$ של ה- $masterLink$ שווה ל- $keyToMaster$ של הסוס

אם הבדיקה צלחה נגדיר את ה- $master$ להיות "הסוס הנוכחי", נוודא שעוד לא ראינו אותו ונמשיך באופן זה עד שנגיע לסוס עם $otherHorseId$.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. אנו מבצעים שני חיפושים בעץ כל הסוסים ב- $O(\log n)$ במקרה הגרוע. לאחר מכן נעבור לכל היותר על כל הסוסים של העדר, כלומר $O(n_{herdId})$ במקרה הגרוע. עדכון שדות הוא $O(1)$. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log n + n_{herdId})$ במקרה הגרוע.

• $can_run_together(int herdId)$

רוצים לבדוק אם הסוסים בתוך העדר יוצרים גרף מכון חסר מעגלים.

נבדוק ש- $herdId$ תקין ושקיים עדר כזה. אם יש סוס אחד בעדר - הצלחנו. אחרת נעבור על העץ של העדר, נבדוק כמה $slaves$ יש לכל סוס ונשמור בשדה $sumOfSlaves$. נגדיר שתי מחסניות (מימשנו לצורך כך מחלקת עזר $HorseStackLIFO$) $bots$ (סוסים שאף אחד לא מצביע עליהם) ו- $tops$ (סוסים שלא מצביעים על אף אחד). אם יש יותר מסוס אחד ב- top נוכל להפסיק (כי אסור שיהיה יותר ממוביל אחד). נגדיר $counter$ שיספור את כל הסוסים שאנחנו עוברים עליהם. ניגש ל- $bots$ ונוציא משם סוס, נגדיר אותו להיות "הסוס הנוכחי", נסמן אותו על ידי שינוי השדה $flagFound$ למספר האיטרציה הנוכחית כדי לדעת שעברנו עליו ומתי זה קרה. נבדוק אם ל"סוס הנוכחי" יש $master$ באופן הבא:

- נוודא שיש לסוס $masterLink$ (ה- $HorseLink$ של ה- $master$)
- נוודא שה- $publicKey$ של ה- $masterLink$ שווה ל- $keyToMaster$ של הסוס

אם הבדיקה צלחה נגדיר את ה- $master$ להיות "הסוס הנוכחי". נבדוק את השדה $flagFound$:

- אם ראינו את הסוס באיטרציה הנוכחית יש מעגל ונכשלו
- אם ראינו את הסוס באחת האיטרציות הקודמות נעצור ונעבור לסוס הבא מ- $bots$

אם סיימנו את האלגוריתם בהצלחה נבדוק אם $counter == n_{herdId}$. אם כן אז עברנו על כל סוסי העדר והם אכן יוצרים גרף מכון חסר מעגלים, לכן הם יכולים לרוץ ביחד. לבסוף נמחק את המחסניות.

סיבוכיות זמן: בדיקת תקינות הקלט היא $O(1)$. מבצעים חיפוש בעץ כל העדרים הלא ריקים ב- $O(\log m)$ במקרה הגרוע, עוברים על העץ של העדר ב- $O(n_{herdId})$ במקרה הגרוע, מגדירים 2 מחסניות ו- $counter$ ב- $O(1)$, מבצעים את האלגוריתם תוך מעבר על כל סוסי העדר ב- $O(n_{herdId})$ במקרה הגרוע ומבצעים השוואה ב- $O(1)$. לפי כך הסיבוכיות הכוללת של פעולה זו היא $O(\log m + n_{herdId})$ במקרה הגרוע.